

Design Engineer Take-Home: Trace Monitor + Slack Cards

We're hiring a **design engineer** to own the end-to-end product experience from design decisions through to shipping the frontend. This exercise is built to show three things at once: that you write **modular, reusable frontend code**, that it's **efficient**, and that you have **genuine design sense** - the ability to match a design language and sweat the details.

You'll build three connected surfaces for an LLM observability product:

1. **A trace explorer** - in the spirit of **Linear's design**: dense, keyboard-friendly, calm, fast.
2. **A Slack card system** - the cards a monitoring bot posts to Slack when something noteworthy happens to a trace. You'll faithfully render Slack's Block Kit format and design the lifecycle of a card.
3. **An overview dashboard** - a focused metrics view across all traces (latency, cost, errors, volume), in the spirit of an observability dashboard.

There is **no mockup**. Owning the visual and interaction design is the job - that's deliberate.

Time budget: plan for **1–2 focused days**. We genuinely don't want you grinding longer. A small, beautifully crafted, well-architected build beats a large sloppy one every time. If you run short, cut breadth, keep craft, and tell us what you cut.

Background

LLM apps emit **traces** - a record of one execution. A trace is a tree of **spans** (a call to a model, a tool, a retriever, a sub-chain). Teams watch these to answer: *what failed, why was it slow, what did it cost?*

When something notable happens - a trace **errors**, or a run gets **negative feedback** - a monitoring **bot posts a card to Slack** so the team sees it without opening the dashboard. That card has a **lifecycle**: it starts as an alert, shows the bot investigating, becomes a triage card with a root cause and actions, and finally collapses to a resolved state. Your Slack surface models exactly that.

Think LangSmith/Langfuse for the data, **Linear for the design**, and Slack's real Block Kit for the cards.

What we give you

- `traces.json` - ~50 traces. Read the `$schema_note` at the top; it documents every field. A trace has `id`, `name`, `status` (`success` | `error` | `running`), `timing`, `totalTokens`, `totalCostUsd`, `tags`, `metadata`, optional `feedback`, and a tree of `spans` linked by `parentId`. **Watch for:** `running` traces have no `endTime`; some traces carry `feedback` (like/dislike + score); errors carry a real `error` message.
- `slack-cards.json` - 6 Slack messages a bot would post, each with a real **Block Kit** `blocks` array. Read its `$schema_note` - it enumerates every block and element type you need to support. The messages reference traces by `traceId` and cover the full lifecycle (alert - investigating -> triage -> resolved, plus a feedback card).

A note on the data: it's modeled on real production traces, which are rarely perfectly clean — expect the occasional gap or inconsistency. Handle it defensively and make sensible calls where the data is imperfect, and note anything you spotted (and how you handled it) in your README.

Load the data however you like (import, `/public`, or a route handler).

Tech stack (required)

- Next.js (App Router) + TypeScript
- Tailwind CSS
- `shadcn/ui` for base components

Use whatever else you'd normally reach for; justify it in your README.

What to build

Part A - Trace explorer (Linear-style)

A **list** of traces and a **detail** view.

- **List:** each row shows name, status, start time, latency, tokens, and cost. Make it dense and instantly scannable - monospace IDs, status as a calm color signal (not a loud pill), tabular numbers. Include **search** by name and **filter** by status. Errors and running traces must read differently at a glance.
- **Detail:** render the **span tree** (respect `parentId` nesting). Selecting a span shows its `input / output` (arbitrary JSON - render it readably), timing, type, and for `llm` spans the model + token/cost breakdown. Show a rolled-up trace summary (total latency / tokens / cost). Handle `error` and `running` spans as designed states, not crashes.

We're looking for the **Linear feel**: restraint, density done tastefully, keyboard affordances, smooth micro-interactions. (Reference: clean dense lists, muted metadata, monospace identifiers, a quietly excellent detail layout.)

Part B - Slack cards (the modularity centerpiece)

Build a **faithful renderer for Slack Block Kit** plus the card lifecycle.

1. **Block renderer.** Given a `blocks` array from `slack-cards.json`, render a message that looks like a real Slack card: `header`, `context`, `section` (body text and/or two-column `fields`), `divider`, and `actions` (buttons with `primary` / `danger` / default styles, and a `static_select` dropdown). You must parse a sensible subset of Slack `mrkdwn`: `*bold*`, `_italic_`, ``code``, code fences, `<url|label>` links, `:emoji:`, and `\n`. **This is the part we read most closely for modular, reusable code** - design it so adding a new block type is a small, isolated change, not a new branch in a giant switch you have to reason about.
2. **Lifecycle.** Show the full card lifecycle for a trace (alert -> investigating -> triage -> resolved). How you present it is your call - a thread, a stepper, a state toggle - but the transitions should feel intentional.
3. **Interactivity.** The action buttons / select should *do* something client-side - e.g. "Approve & merge" advances the card to resolved, "Mute" dismisses it, the merge-strategy select records a choice. No real backend needed; model the state cleanly.

Part C - Overview dashboard (dense data-viz)

A small **metrics overview** across all traces — the screen a team lead opens each morning, in the spirit of an observability dashboard. Keep it focused; this is about **legible, honest data-viz**, not chart quantity. Include:

- **Headline stat tiles** — total traces, total cost, total tokens, overall error rate, and p50 / p95 latency.
- **A latency distribution** (histogram) with **p50 / p95** marked.
- **Cost by model** (the dataset spans several models at different price points).
- **At least one time-series** over the ~7-day window — e.g. trace volume, error rate, or cost per day.
- One status / feedback breakdown of your choice (success vs error; 👍 vs 👎).

Use any chart library (Recharts, visx, ...) or hand-roll with SVG — tell us why. Charts must have clear units/axes, sensible empty states, and a visual language coherent with the rest of the app. Derive every number from the data (don't hardcode).

Connect the two

From a trace (especially a failed or thumbs-down one), let the user **preview the Slack**

card the bot would post about it - the two surfaces should feel like one product, not two demos bolted together.

Make it feel real

Designed **loading**, **empty**, and **error** states. Responsive enough to not break on a smaller screen. Sensible keyboard/click affordances.

Stretch goals (only if you have time)

Pick at most one or two - tiebreakers, not requirements.

- A **card composer / Block Kit playground**: edit the blocks JSON and see the card update live.
 - **Deep linking** (open trace / selected span / open card live in the URL).
 - **Command palette** (⌘K) and full keyboard navigation.
 - A **timeline/waterfall** of spans (parallel spans shown concurrently).
 - **Light + dark** themes, both excellent.
-

What we're evaluating

In rough priority order. The first three are Ritvik's headline rubric; the rest are how we tell candidates apart.

1. **Modular UI / frontend code** - especially the Block Kit renderer. Is it a clean component system (a block→component registry, composable, easy to extend) or a tangled mega-component? Clear separation of data-shaping from presentation.
2. **Efficient code** - derived data computed once, not recomputed every render; no needless re-renders; sensible handling of the trace list and trees; no obvious waste. Clean, not over-engineered.
3. **Design sense** - does it look and feel like a real product in the Linear register? Typography, spacing, color-as-signal, hierarchy, motion, restraint. Would you ship it?
4. **Fidelity & attention to detail** - how close is the Slack card to a real Slack card? Are `mrkdwn`, emoji, button styles, and the two-column fields right? Details are where craft shows.
5. **Data-viz design** - is the dashboard legible and honest? Right chart per question, clear units/axes, p50/p95 not faked as an average, coherent with the rest of the app.
6. **Type safety & data modeling** - clean types for traces/spans and for the block union (a discriminated union, ideally); no stray `any`; validation at the data boundary.
7. **Robustness** - `running` / `error` states, long content/overflow, empty states, unknown

block types degrade gracefully.

8. **Ownership & communication** - a README that explains your design intent, the trade-offs you made, and what you cut and why.

We are **not** grading: a real backend, auth, tests-for-tests'-sake, or pixel-perfect mobile. A focused, polished build beats a sprawling one.

Submission

Both are **required** - send us both:

- **A deployed link** - host it anywhere (Vercel is perfect and free; Netlify / Cloudflare Pages are fine). We want to click into the live thing and feel it.
- **A public GitHub repository** with full source and commit history (please don't squash into one commit - we look at how you work).

Include a **README** in the repo covering:

- How to run it (`install` + `dev`).
- **Your design thinking** - the look/feel you went for, your component architecture (especially the Slack renderer), and why. This matters as much as the code.
- Key technical trade-offs, and what you'd refine or build next.

If anything is ambiguous, make a reasonable call and document it - that judgment is part of what we're assessing. Have fun with it